

VSR Authentication Integration Report

1. Problem

While deploying UOIS in the VSR (Verified Secure Realm) infrastructure, we identified a critical security gap: the existing authentication middleware accepts any valid JWT regardless of its origin. An attacker (or foreign OAuth server) can issue a token for a Vietnamese citizen, and UOIS will process it — effectively delegating identity authority to external servers and forcing citizen credential data across borders.

2. Root Cause

UOIS uses `BasicAuthBackend4Phase` which validates tokens via RS256 against a configurable OAuth public key endpoint (`JWTPUBLICKEY`). The validation is purely cryptographic — if the signature matches, the token is accepted. There is no check on the `iss` (issuer) claim. This works in a single-tenant deployment where the OAuth server is trusted, but in a multi-realm or cross-border context, a foreign server holding any user's email/password can mint a valid-looking RS256 token and access UOIS resources.

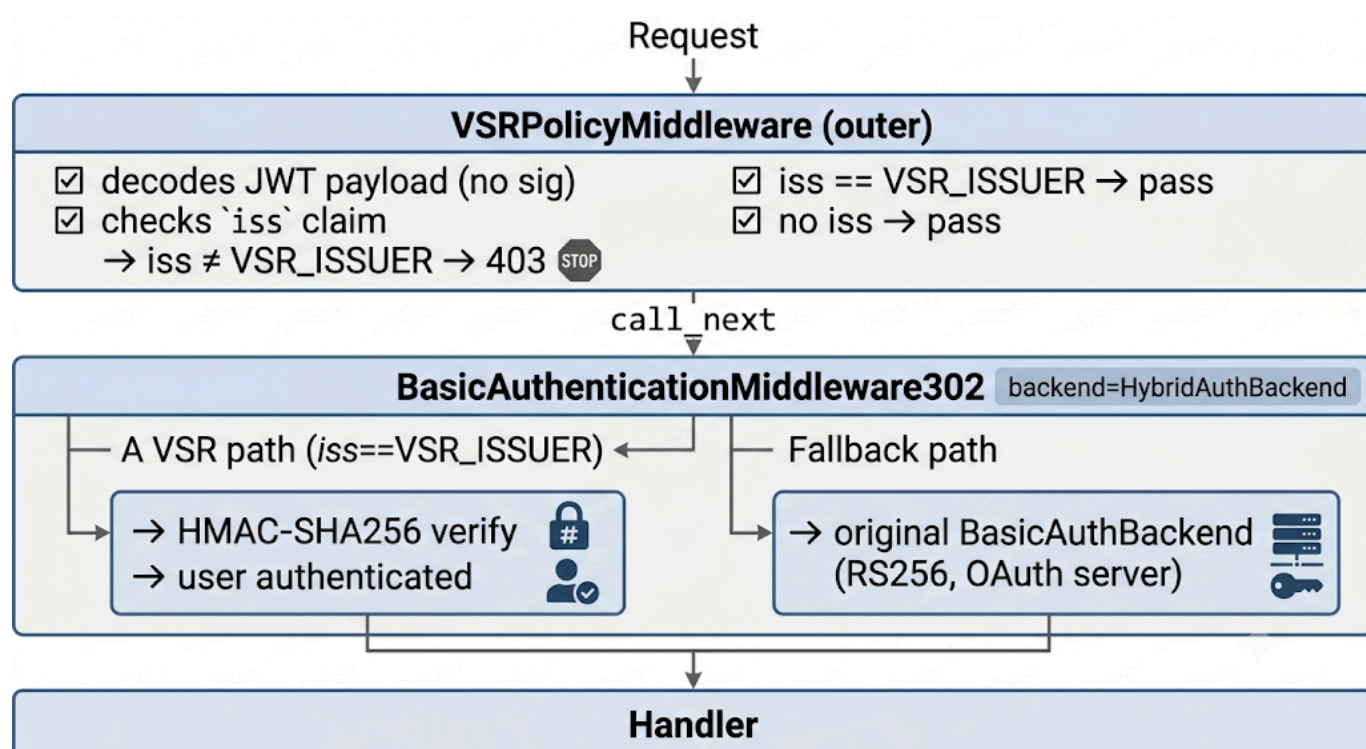
This is specifically a concern for Vietnam because:

- Citizen identity data is legally required to remain within national boundaries.
- Foreign-issued tokens that pass auth effectively export credential verification across borders.
- The existing auth stack has no policy layer to distinguish between a local and foreign issuer.

3. Solution

We implemented a two-layer middleware stack that enforces issuer policy **without modifying any original UOIS auth code**. The original `BasicAuthBackend4Phase` remains untouched.

Architecture:



4. Results

Token Flows (DEMO=False):

Token Type	VSRPolicyMiddleware	HybridAuthBackend	Result
VSR HS256 (<i>iss=VSR_ISSUER</i>)	passes	HMAC verify → success	200
Google RS256 (<i>iss=accounts.google.com</i>)	403	—	403
No token	passes	fallback → redirect to /oauth/login2	302
VSR + bad HMAC	passes	HMAC fail → AuthenticationError	302
OAuth RS256 (<i>iss=localhost</i>)	passes (no VSR_ISSUER)	fallback → RS256 verify → success	200

Test Results:

The output result is expected from the table above. The test scripts can be found in `scripts/test-*.sh` and can be run individually or all at once with `bash scripts/test-all.sh`.

Test to make sure VSR token with correct issuer and signature is accepted (DEMO=False):

```

creating sub app facilities with setup {'file': 'facilities.html', 'title': 'Učebny', 'authors': 'jokachu', 'markdownfile': 'facilities.md'}
creating sub app granting with setup {'file': 'granting.html', 'title': 'Akreditace / garance', 'authors': 'white.002', 'markdownfile': 'granting.md'}
creating sub app lessons with setup {'file': 'lessons.html', 'title': 'Plány studia', 'authors': 'kristl63', 'markdownfile': 'lessons.md'}
creating sub app surveys with setup {'file': 'surveys.html', 'title': 'Studentské hodnocení', 'authors': 'DT', 'markdownfile': 'surveys.md'}
creating sub app projects with setup {'file': 'projects.html', 'title': 'Projekty', 'authors': 'LZ', 'markdownfile': 'projects.md'}
creating sub app publications with setup {'file': 'publications.html', 'title': 'Výsledky', 'authors': 'K4rleak', 'markdownfile': 'publications.md'}
creating sub app forms with setup {'file': 'forms.html', 'title': 'Žádosti', 'authors': 'nikdo3', 'markdownfile': 'forms.md'}
creating sub app importer with setup {'file': 'importer.html', 'title': 'Importer', 'authors': 'Hrbolek', 'markdownfile': 'importer.md'}
creating sub app performance with setup {'file': 'performance.html', 'title': 'Performance test', 'authors': 'Hrbolek', 'markdownfile': 'performance.md'}
creating sub app users with setup {'file': 'users.html', 'title': 'Uživatelé', 'authors': 'Hrbolek', 'markdownfile': 'users.md'}
creating sub app exams with setup {'file': 'exams.html', 'title': 'Zkoušky', 'authors': 'many', 'markdownfile': 'exams.md'}
creating sub app form with setup {'file': 'form.html', 'title': 'DEMO', 'authors': ':)', 'markdownfile': 'form.md'}
creating sub app debug with setup {'file': 'debug.html', 'title': 'Debug', 'authors': 'Hrbolek', 'markdownfile': 'debug.md'}
first user {'id': '51d101a0-81f1-44ca-8366-6cf51432e8d6', 'email': 'john.newbie@world.com'}
using proxy http://10.0.2.27:33001/gql
DIST_DIR /home/aki/JWT/_uois/server/htmls/dyn
INFO: Started server process [40852]
INFO: Waiting for application startup.
2026-06-04T10:07:10.574 INFO: starting engine for "sqlite+aiosqlite:///test_vsr.db makeDrop=False"
BaseModel.metadata.create_all finished
2026-06-04T10:07:10.581 INFO: initializing system structures
2026-06-04T10:07:10.581 INFO: all done
2026-06-04T10:07:10.581 INFO: Inserting users into DB
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
serving app debug.html from /home/aki/JWT/_uois/server/htmls/debug.html
INFO: 127.0.0.1:60376 - "GET /debug/ HTTP/1.1" 200 OK
INFO: 127.0.0.1:60392 - "GET /debug/ HTTP/1.1" 403 Forbidden
# BEGIN #####
http://localhost:8000/ Address(host='127.0.0.1', port=80394), Headers({'host': 'localhost:8000', 'user-agent': 'curl/8.20.0', 'accept': '*/*'}), {}
/debug/
get_jutsources None
INFO: 127.0.0.1:60394 - "GET /debug/ HTTP/1.1" 302 Found
INFO: 127.0.0.1:60404 - "GET /debug/ HTTP/1.1" 302 Found
(.venv) [aki@cache05-Aki] - [~/JWT/_uois] - [Thu Jun 04, 22:07]
[git:(latest)] bash scripts/test-all.sh
===== 1. VSR token (valid HMAC) =====
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <link rel="icon" type="image/svg+xml" href="/vite.svg" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>UOIS App Debug</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="http://localhost:5173/src/main.jsx"></script>
</body>
</html>
HTTP 200

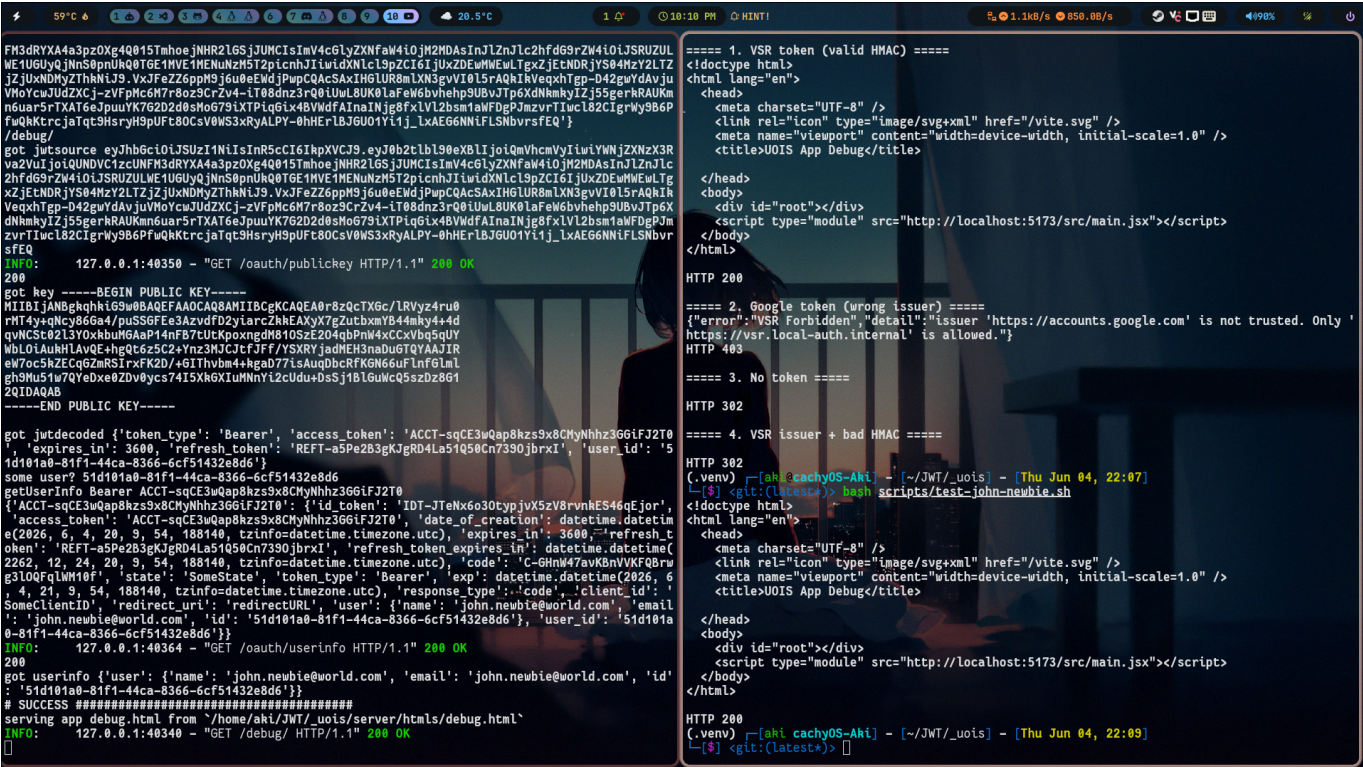
===== 2. Google token (wrong issuer) =====
{"error": "VSR Forbidden", "detail": "issuer 'https://accounts.google.com' is not trusted. Only 'https://vsr.local-auth.internal' is allowed."}
HTTP 403

===== 3. No token =====
HTTP 302

===== 4. VSR issuer + bad HMAC =====
HTTP 302
(.venv) [aki@cache05-Aki] - [~/JWT/_uois] - [Thu Jun 04, 22:07]
[git:(latest)]

```

Extra test to make sure the original OAuth flow is unaffected (DEMO=False):



5. Source Files

Modified Files:

File	Change
<code>_uois/vsr/vsr_auth.py</code>	Created — core VSR logic, <code>VSRPolicyMiddleware</code> , <code>HybridAuthBackend</code>
<code>_uois/server/main-vsr.py</code>	Created - a copy of <code>_uois/server/main.py</code> with 5 sub-apps: replaced stacked auth with <code>HybridAuthBackend</code> + <code>VSRPolicyMiddleware</code>
<code>_uois/scripts/test-*.sh</code>	Created — test scripts for all 4 scenarios
<code>_uois/VSR_INTEGRATION.md</code>	Created — full technical documentation

Source Code:

All changes have been published to GitHub: https://github.com/getbricked/_uois